

Project 1:
**Cloud-Native Deployment Dependency and Priority
Scheduling System**
Using Python

Ashish Mahajan

ID: 005048542

University of the Cumberlands

MSCS 532-B01: Algorithms and Data Structures

Project Phase 1 Deliverable 1

Dr. Michael Solomon

July 12, 2026

GitHub repository: https://github.com/AshishM26/MSCS532_Project_AM

2026 Summer - Algorithms and Data Structures (MSCS-532-B01)

Cloud-Native Deployment Dependency and Priority Scheduling System

Application Context and Problem Definition

Cloud-native applications contain services and infrastructure components that must often be deployed in a valid order. A network layer may be required before secrets are loaded, and a database may need validation before an application programming interface starts. Unclear dependencies can cause invalid sequencing, blocked work, or circular relationships.

This project implements foundational data structures for a generic deployment-dependency and priority-scheduling system. Seven nonproprietary services are modeled: network-foundation, secrets-service, database-service, backend-api, frontend-service, monitoring-agent, and observability-dashboard. The Phase 1 scope is deliberately limited to an in-memory, single-process Python model.

The design separates three concerns. A directed graph determines deployment readiness; a dictionary-backed registry stores service state, and a binary-heap priority queue determines urgency among ready services. This separation is the project's central insight because each structure is assigned the responsibility that best matches its behavior, fostering trust in the system's clarity and reliability.

Directed acyclic graphs are commonly used for workflows with precedence constraints. Topcuoglu et al. (2002) model dependent tasks as directed acyclic graph and separate task ranking from resource assignment. Deelman et al. (2015) similarly describe graph-based workflows that are later mapped to execution resources. These findings support the project's distinction between dependency correctness and scheduling priority.

The practical rule is that priority cannot override readiness. A critical backend service cannot be deployed before its database or secrets prerequisites are met. The graph first identifies eligible services, and the heap ranks only that safe subset.

2026 Summer - Algorithms and Data Structures (MSCS-532-B01)

Data Structure Design and Rationale

Deployment Graph uses a directed adjacency list. Services are vertices, and an edge from A to B means A is a prerequisite of B. An adjacency list is preferable to an adjacency matrix because cloud-service graphs are usually sparse. It requires $O(V + E)$ space instead of $O(V^2)$, where V is the number of services and E is the number of dependencies.

Cycle detection raises a `DependencyCycleError` when a cycle is detected. However, it also provides clear guidance and recovery steps to help users resolve such issues, enhancing system robustness and user support.

`ServiceRegistry` uses a Python dictionary keyed by the service identifier. It stores the environment, version, status, owner, and priority. Lookup, insertion, update, and removal are $O(1)$ on average, compared with $O(n)$ scanning in a list. Required fields are validated, and deep copies protect internal records from unintended external mutation. The theoretical worst-case running time of the dictionary remains $O(n)$ under severe collisions.

`DeploymentPriorityQueue` uses a binary min-heap (heapq) with priorities ranging from 1 (critical) to 4 (low). Peek is $O(1)$, while enqueue and dequeue are $O(\log n)$. A sorted list was rejected because maintaining order after insertion can require $O(n)$ work. An insertion counter preserves deterministic order for equal priorities.

Cloud workflow scheduling may eventually consider deadlines, costs, and resources in addition to dependencies (Abrishami et al., 2013). Phase 1 intentionally uses fixed priorities, leaving dynamic ranking for later optimization.

2026 Summer - Algorithms and Data Structures (MSCS-532-B01)

Python Implementation and Validation

The project was implemented in Python 3.11.1 using only the standard library. Separate modules implement `DeploymentGraph`, `ServiceRegistry`, and `DeploymentPriorityQueue`. A demonstration script integrates the structures, and unit tests validate their behavior independently.

The demonstration registered seven services, retrieved metadata for the database services, updated their status from pending to validated, and reported prerequisites and dependents. The valid graph returned `False` for cycle detection and generated a topological order that preserved every prerequisite relationship. Independent branches may appear in different positions so that multiple topological orders can be corrected.

Priority-aware scheduling inserted only currently eligible tasks into the heap. Network-foundation was selected first because it was the only ready service at the time. Upon completion, additional services became eligible and were selected in order of priority. The Backend API was not eligible until its required services were processed, demonstrating that dependency correctness is evaluated before urgency.

The standalone queue example returned equal-priority services in insertion order. A separate invalid graph introduced a cycle, which was detected and handled safely with a clear message rather than an unexpected crash.

Fourteen unit tests were executed with `python3 -m unittest discover -s tests -v`. They covered graph construction, duplicate edges, cycle detection, topological sorting, metadata operations, priority and tie handling, invalid priorities, and empty-queue behavior. All tests passed without failures or errors. The 0.001-second local runtime confirms completion and demonstrates the system's reliability and correctness.

2026 Summer - Algorithms and Data Structures (MSCS-532-B01)

Critical Evaluation, Limitations, and Future Optimization

No single structure is sufficient for dependency-aware scheduling. The graph represents readiness; the registry provides state access, and the heap selects urgent eligible work.

Phase 1 stores data only in memory and does not execute deployments, persist state, coordinate concurrent schedulers, authenticate users, retry failures, or recover distributed work.

Discuss how the current $O(V + E)$ algorithms perform on larger, sparse graphs and outline planned scalability improvements to address potential concerns about real-world applicability, encouraging confidence in future system expansion.

Caching repeated readiness results is the second priority. If the prerequisites have not changed, recalculating eligibility is a waste of work. A future engine could cache readiness and invalidate only affected downstream services. This exchanges additional memory and invalidation complexity for fewer repeated traversals.

Elaborate on how dynamic priority updates, considering factors such as age or security risk, will be integrated into the system and the expected benefits for scheduling flexibility.

Workflow research supports a staged progression from dependency modeling to execution and monitoring (Yu & Buyya, 2005; Deelman et al., 2015). Future work should benchmark scale, add caching and dynamic priorities, then evaluate persistence or CI/CD integration.

2026 Summer - Algorithms and Data Structures (MSCS-532-B01)

References

- Abrishami, S., Naghibzadeh, M., & Epema, D. H. J. (2013). Deadline-constrained workflow scheduling algorithms for Infrastructure as a Service clouds. *Future Generation Computer Systems*, 29(1), 158–169. <https://doi.org/10.1016/j.future.2012.05.004>
- Deelman, E., Vahi, K., Juve, G., Rynge, M., Callaghan, S., Maechling, P. J., Mayani, R., Chen, W., Ferreira da Silva, R., Livny, M., & Wenger, K. (2015). Pegasus is a workflow management system for science automation. *Future Generation Computer Systems*, 46, 17–35. <https://doi.org/10.1016/j.future.2014.10.008>
- Topcuoglu, H., Hariri, S., & Wu, M.-Y. (2002). Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems*, 13(3), 260–274. <https://doi.org/10.1109/71.993206>
- Yu, J., & Buyya, R. (2005). A taxonomy of scientific workflow systems for grid computing. *ACM SIGMOD Record*, 34(3), 44–49. <https://doi.org/10.1145/1084805.1084814>
- GitHub repository: https://github.com/AshishM26/MSCS532_Project_AM
-

